So You Want to Write Some Client-Side Code

“I need a website,” the client said.

“Great,” you think. Ruby on Rails is a solid way to go. Version 7.0 just came out. It’s still the best way for a small team to be as productive as a big team. You are ready for this. You start thinking of estimates and modeling data structures…

“I want it to look cool, with lots of stuff moving around, and be extremely interactive,” the client added.

“Ugh,” you think. That brings in JavaScript. And with it, a whole lot of decisions. What language? There’s not just JavaScript, but a host of languages that compile to JavaScript: TypeScript, Elm, ClojureScript. What framework? There are dozens: React, Vue, Ember, Hotwire, Svelte, Preact, and on and on. How to package the code and CSS? Should you use the existing Rails asset pipeline, or Propshaft, or jsbundling, or Webpacker? What about that new Hotwire thing the Rails team has been going on about?

Suddenly you are overwhelmed by the added complexity.

Although it’s primarily a server-side tool, Ruby on Rails offers a lot of support for client-side code. Rails version 7.0 has tools that help you interact with the JavaScript ecosystem to build an exceptional front-end experience. In this book, you’ll learn how you can enhance the user experience of a standard Rails application using front-end tools from the Rails ecosystem (Hotwire, Stimulus, Turbo, and jsbundling) and tools from the JavaScript ecosystem (esbuild,